# Encryption (confidentiality)

## TBK SPEC 10:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 10:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

# Contents


Foreword ____ 4

1. Model ____ 4

2. Key model (locked) ____ 4

3. Memory discipline ____ 5

4. Serving ____ 5

5. PQC readiness (locked) ____ 6

6. Cyber/admin algebra (why this changes the game) ____ 6

7. CLI surface (SHIPPED v1) ____ 7

4a. The wire format (normative — the byte layout an independent implementation must match) ____ 7



© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com


© 2026 The Tebako Project

# Foreword

Normative specification of encrypted volumes: selective, in-memory-only decryption. Status: v1 SHIPPED (roadmap 37: `EncBackend` + `tfs encrypt/decrypt/mount`, SUITE-1; PQC suites, metadata hiding and merkle verify-on-read remain PLANNED). Encryption is per-image **opt-in**, never default.

# 1. Model

Encryption is a **stacking transform** like COW, not a format: `EncBackend { base: dyn Backend, key_source }` — reads decrypt on demand, in memory only; plaintext blocks never touch disk. Compose freely: `Enc(COW(base))`, `COW(Enc(base))`, nested encapsulated encrypted images. Per the transforms law (spec 00): `EncBackend` lives ONLY in the Rust TFS — backends and dwarfs-t stay ignorant of it.

**Primitive:** OpenPGP via rnp-rs (AES-256/SHA-2; botan backend). Encrypt AFTER compress (ciphertext doesn't compress; dwarfs blocks are individually compressed → individually encryptable). Optional padding to blur per-block sizes.

# 2. Key model (locked)

- **DEKs (data keys)** encrypt content blocks. A DEK IS a persisted OpenPGP session key — rnp's native wrap/unwrap (PKESK packets to N recipients) is the envelope; no custom crypto anywhere.


© 2026 The Tebako Project

- **Identity lives at the PLAINTEXT layer.** Manifests carry TWO digests (spec 03 §2.1): the plaintext TREE hash (semantic identity — CAS addressing, dedup, sign/verify, audit-without-access) and the ciphertext blob hash (transport identity). Encryption is a per-audience transform; CAS never keys content-identity to any one wrapping.
- **Session keys are grant envelopes.** Sharing = wrapping the DEK to a recipient (ephemeral ECDH per OpenPGP) — offline, accountless, auditable. A wrapped DEK + manifest digest IS a capability: possessing it grants exactly that subtree, nothing else.
- **Revocation is prospective:** re-issue the manifest without the recipient (no new envelopes), or rotate the DEK for FUTURE versions (full re-encrypt, only on suspected DEK compromise). You cannot un-read what was read — documented honestly.
- **Hierarchical derivation:** subtree keys derive from a root DEK via HKDF with the path as info — K(/a/b) grants /a/b/** but not /a/c or /a (one-way). Sharing a directory subtree = handing over its derived key. Layered composites: per-layer DEKs for coarse audiences, HKDF subtrees for fine-grained ones.
- **Composition rules:** JOIN/squash of layers with different DEKs = authorized re-encryption under a new joined key, OR preserved per-layer key domains in the joined manifest (lazy join). Encapsulated images keep INDEPENDENT key domains — outer may be readable while inner stays sealed (transport vs payload audiences).
- **Selective disclosure:** per-directory/per-layer recipient mapping in the manifest → ONE artifact, N audiences; each key opens only its slice.
- **Metadata hiding option:** encrypt the metadata block as a unit — structure/filenames invisible until keyed.

# 3. Memory discipline

Decrypted blocks live in heap buffers that are `mlock`'d + zeroized on free. Keys via agent / passphrase / PKCS#11 hardware — never written to disk, never in swap. Confidential-computing memory (SGX/SEV/TDX) is a documented future tier, not v1.

# 4. Serving

Prefer ciphertext end-to-end (clients decrypt locally; the server/CDN never sees plaintext). Session wrapping per online client only when the server is trusted with keys. Block-access patterns leak regardless (PIR out of scope) — documented.


© 2026 The Tebako Project

# 5. PQC readiness (locked)

- Algorithm agility from day one: every manifest/envelope/signature carries a CIPHER-SUITE id from a versioned registry — nothing hardcoded. Unknown ids fail with a named error, never a guess.
  - SUITE-1 = X25519 + Ed25519 + AES-256-GCM + SHA-256 (classical).
  - SUITE-2 = HYBRID: ML-KEM-1024+X25519 composite KEM, ML-DSA-87+Ed25519 composite signatures, AES-256-GCM, SHA-384 — secure if EITHER family holds (the transition form).
  - SUITE-3 = SLH-DSA (hash-based signatures) maximal-conservatism tier.
- HNDL: long-lived encrypted images SHOULD use hybrid envelopes NOW. AES-256 keeps ~128-bit post-Grover strength; the merkle/hash layer is PQC-safe by construction. Signatures are the migration surface, envelopes the confidentiality surface — both suite-versioned, so migration is additive.
- Implementation path: botan 3 (rnp's backend) via rnp-rs PQC tracking + OpenPGP PQC drafts; recipients may hold classical AND PQC keys (multi-recipient envelopes are native).

# 6. Cyber/admin algebra (why this changes the game)

- Object security > channel security: artifacts self-protect over ANY transport/mirror (untrusted CDN, USB, email).
- Leak containment: disk snapshots/caches hold ciphertext only.
- Crypto-shredding / kill-switch: destroy a data key (or re-issue the manifest without a recipient) → content unrecoverable everywhere, provably — no mirror-purge problem.
- Immutable golden bases + auditable COW deltas (the overlay IS the change record) + signed provenance = compliance-grade fleet management.
- Honest risks: access-pattern side channels on serve; key-agent UX is the hard part; mlock/zeroize is the baseline, not a panacea.


© 2026 The Tebako Project

# 7. CLI surface (SHIPPED v1)

`tfs encrypt / decrypt / mount --recipient / --key` — encryption at image-creation time through the same Writer path; mount requires the recipient key; wrong key → named EKEY-class error, never garbage. SHIPPED (roadmap 37): `tfs encrypt` (root grant `--recipient`, subtree grants `--subtree <path>=<pubkey>`; `--rewrap --key` rotates grants to a new recipient set with the bulk ciphertext byte-identical), `tfs decrypt` (plaintext to a tar stream — never a staging tree), `tfs mount --key` (the unlock/grant report; FUSE/serve mounts remain spec-11 §6 PLANNED). `tfs sign` is spec 09's surface. The v1 crypto construction (per-block AES-256-GCM, HKDF-SHA256 subtree keys, PKESK DEK envelopes, the `/__tpkg__/envelopes.yaml` grant manifest) is documented normatively in `crates/tfs/src/backends_enc.rs` and `crates/tpkg/src/envelope.rs`.

# 4a. The wire format (normative — the byte layout an independent implementation must match)

Directory structure, names, symlink targets and the `/__tpkg__/` metadata directory stay plaintext (metadata hiding is this spec's deferred option); every other regular file's CONTENT is replaced by its encrypted form:

**FIGURE 1**

```
offset  size  field
     0     8  magic "tfsenc01"
     8     8  u64le plaintext_size
    16     –  per block i (block = 4096 bytes; full-block
slot 4112):
                ciphertext_i (block_len bytes) || GCM tag_i
(16 bytes)
```

Per block, AES-256-GCM with:

- key = the file key `FK(path)` (below),
- nonce = `u32le(0) || u64le(block_index)` (12 bytes) — unique per (key, block): file keys are unique per file, block indexes never repeat within a file, and images are immutable, so no (key, nonce) pair ever seals two different plaintexts. A CHANGED


© 2026 The Tebako Project

tree's re-encryption MUST rotate the DEK (§2's prospective rotation), which re-keys every nonce domain.

- AAD = `"tfsenc01" || u64le(plaintext_size) || u64le(block_index)` — binds each block to its file size and position (blocks cannot be relocated, truncated away silently, or transplanted between files of different sizes).

Keys: one root DEK per image (32 random bytes), with HKDF-SHA256 path keys for selective disclosure (§2):

**FIGURE 2**

```
K("")            = DEK                          (the image
root key)
K(dir/name)      = HKDF-SHA256(ikm = K(dir),  salt = none,
                               info = "tfs-enc-1/dir\0"  ||
name)
FK(dir/file)     = HKDF-SHA256(ikm = K(dir),  salt = none,
                               info = "tfs-enc-1/file\0" ||
name)
```

One-way per level: K(/a/b) opens /a/b/** but not /a/c or /a. Sharing a subtree = wrapping THAT subtree's key to the recipient. The DEK (or a subtree key) is wrapped to recipients as OpenPGP PKESK packets and recorded in `/__tpkg__/envelopes.yaml` (tpkg's EnvelopeManifest); the payload manifest's `encryption.parts` reference grants by id and NEVER carry keys (spec 03 §2.1).


© 2026 The Tebako Project